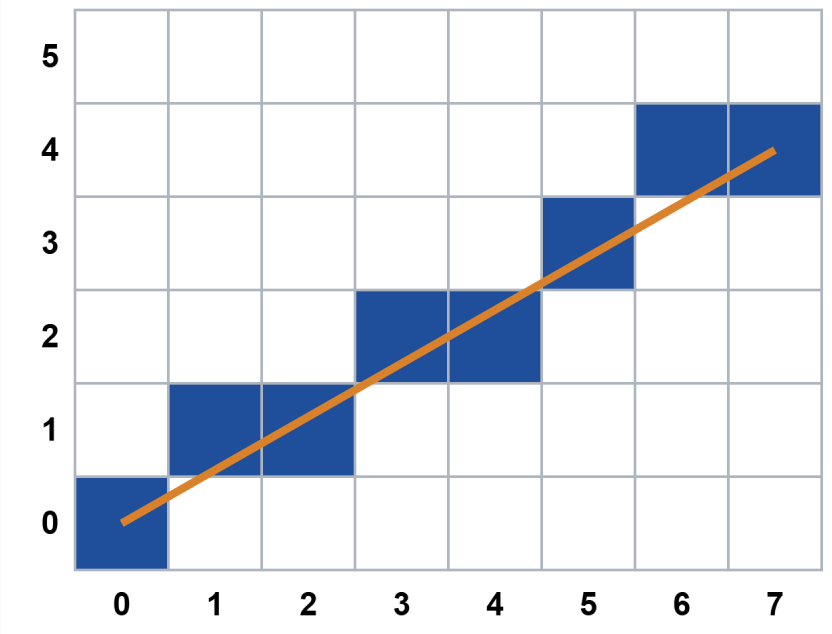
0613-403 · Week 1 · Lecture 2

Colour, Display Devices and Scan Conversion

Ekramul Haque Amin

Lecturer, AI and Data Science

Dhaka International University



Last class in one slide

Pixel

The smallest dot on the screen; each shows one colour

Resolution

Pixels across × pixels down, e.g. 1920 × 1080

Frame buffer

Memory holding the colour of every pixel

Raster vs vector

Pixels vs shapes defined by maths

Plan for this lecture

Part 1

Colour

- RGB model
- Bits per pixel
- Direct coding
- Lookup table

Part 2

Devices

- CRT and raster scan
- Random scan
- Storage tube
- Input and output

Part 3

Scan conversion

- Points
- Direct method
- Its problems
- Aliasing

Part 1

Colour

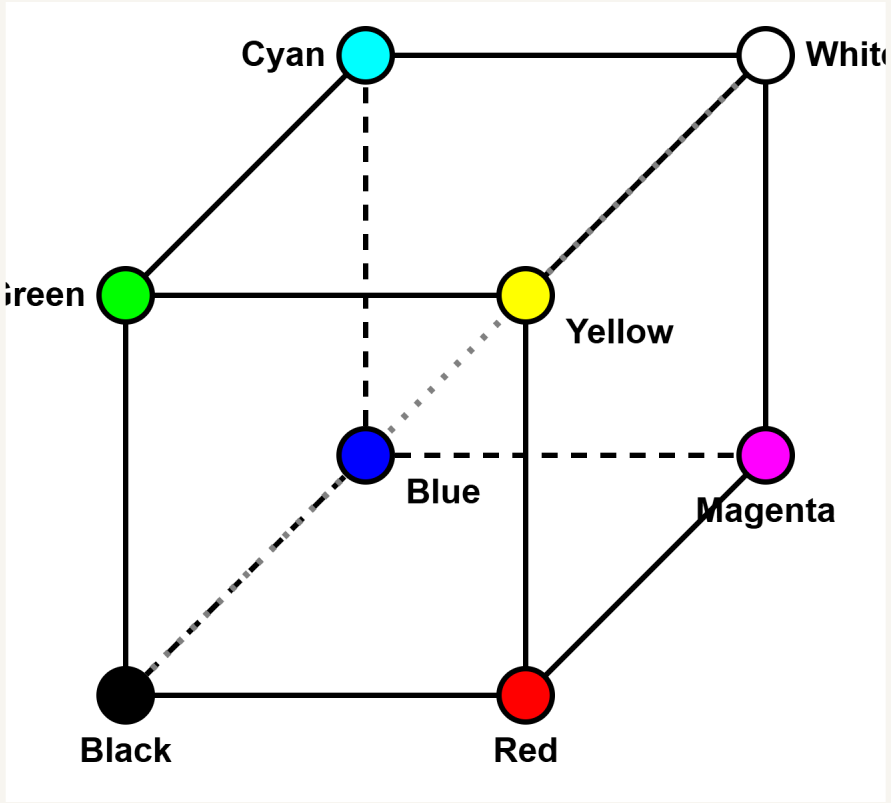
How a computer stores and shows colour

The RGB colour model

Every colour on a screen is a mix of Red, Green and Blue light. Each part is a number from 0 (off) to 255 (full).

Red (255, 0, 0)	**Green** (0, 255, 0)	**Blue** (0, 0, 255)
Yellow (255, 255, 0)	**Cyan** (0, 255, 255)	**Magenta** (255, 0, 255)
Black (0, 0, 0)	**Grey** (128, 128, 128)	**White** (255, 255, 255)

The RGB colour cube



- Three axes: red, green and blue
- Black (0, 0, 0) is at the origin
- White (255, 255, 255) is the opposite corner
- The dotted diagonal holds all the greys
- Every colour is one point inside the cube

Bits per pixel decide how many colours

Bits per pixel	Colours	Name
1	2	Black and white
4	16	Early PCs
8	256	Colour map / greyscale
16	65,536	High colour
24	16,777,216	True colour

Rule

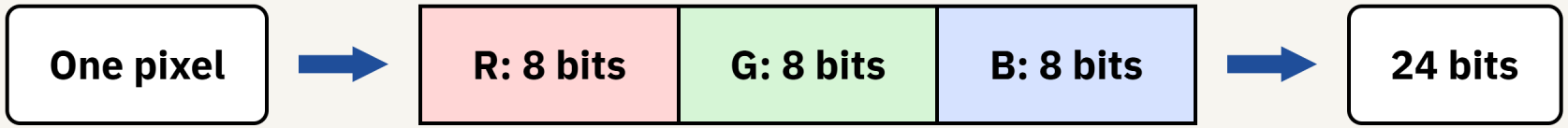
$$\text{colours} = 2^n$$

n = bits per pixel

True colour: 8 bits each for R, G and B = 24 bits

Storing colour 1: direct coding

The frame buffer stores the full red, green and blue values for every pixel.



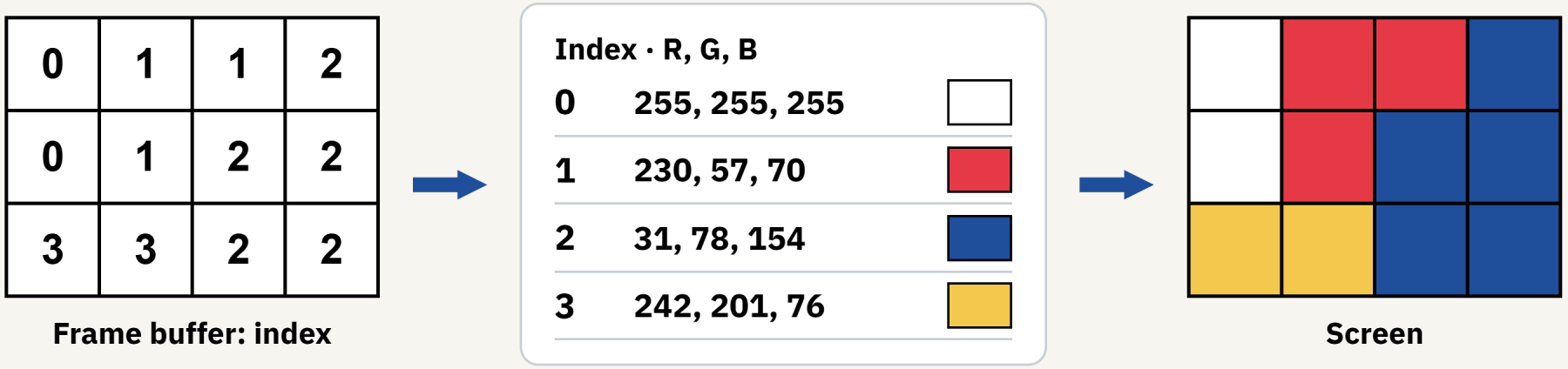
Advantage

Any of 16.7 million colours can be used anywhere, at the same time.

Disadvantage

Needs a lot of memory: 3 bytes for every pixel.

Storing colour 2: lookup table (colour map)



Each pixel stores only a small index. The table turns the index into the real RGB colour.

Worked example: which needs less memory?

Screen: 1024 × 768 = 786,432 pixels

Direct coding, 24 bits

786,432 × 3 bytes

= 2,359,296 bytes

≈ 2.25 MB

16.7 million colours at once

Lookup table, 8 bits

786,432 × 1 byte = 786,432 bytes

+ table: 256 × 3 = 768 bytes

≈ 0.75 MB

256 colours at once, chosen from 16.7 million

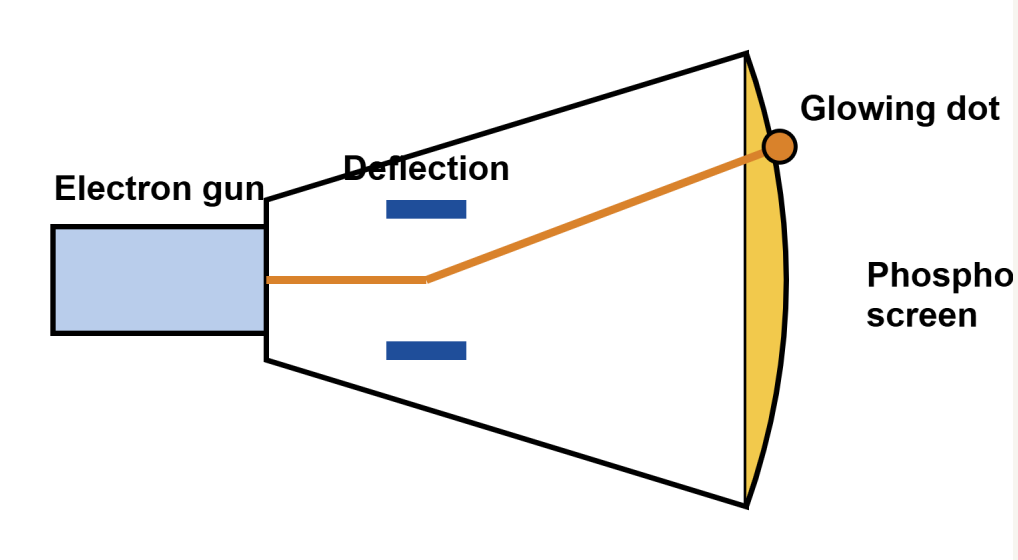
A lookup table saves about two-thirds of the memory, but limits how many colours appear together.

Part 2

Display devices

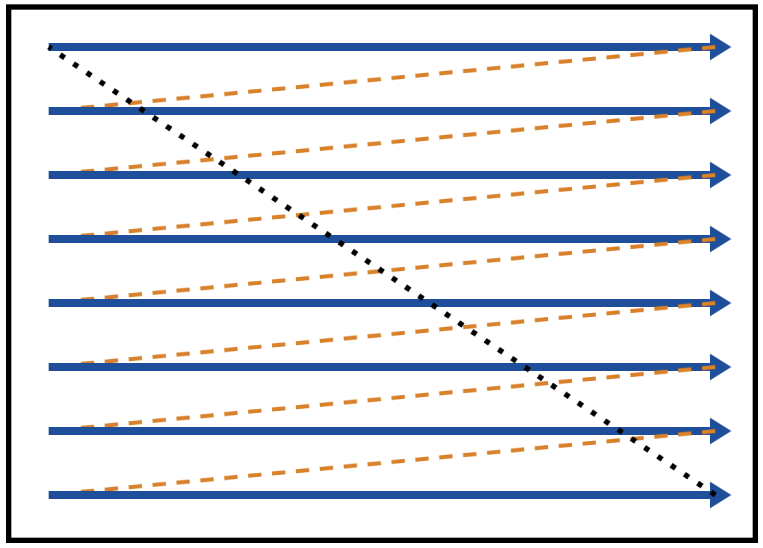
How a picture is drawn on a screen, and how we give input

How a CRT screen works



1. The electron gun fires a beam of electrons
2. Deflection bends the beam to any point
3. Where the beam hits, the phosphor glows
4. The glow fades quickly, so the picture must be redrawn again and again: refresh

Raster scan display



Blue lines: the beam sweeps each row left to right, top to bottom, reading the frame buffer.

Orange dashes: horizontal retrace, the beam returns to start the next row.

Black dots: vertical retrace, back to the top after the last row.

Refresh rate: the whole screen is redrawn 60 times a second or more.

Interlacing: draw odd rows, then even rows, to reduce flicker.

Random scan (vector) display

How it works

- The beam moves straight from one end point to the other, drawing lines directly
- A display list of line commands is redrawn again and again
- No pixel grid

Good and bad

- Very smooth, sharp lines with no jaggies
- Cannot fill areas or show realistic photos
- Used in early CAD systems and oscilloscopes

Storage tube display (DVST)

Direct-View Storage Tube: the picture is stored inside the tube, so it stays on screen without refreshing.

Advantages

- No refresh needed, so no flicker
- Can show very complex pictures
- Cheaper than early refresh displays

Disadvantages

- Cannot erase part of the picture; the whole screen must be cleared
- No colour, low contrast
- Not suitable for animation
- Now obsolete

Comparing the three displays

	Raster scan	Random scan	Storage tube
Draws	Row by row, every pixel	Lines between end points	Lines, stored in the tube
Refresh	Yes, 60+ times a second	Yes, from the display list	No
Colour and filled areas	Yes	Limited	No
Erase part of picture	Yes	Yes	No, whole screen only
Used today	Yes: all monitors and TVs	Rarely	No

Input devices

Keyboard and mouse

Text, commands and pointing

Trackball and joystick

Pointing and control in games and simulators

Digitizer (tablet)

Trace a drawing with a stylus; gives exact (x, y) positions

Light pen

Points at the screen and senses its light; used by Sketchpad

Touch screen

Point directly with a finger

3D input

VR controllers and data gloves track hand position

Hard-copy and 3D viewing devices

Printers

- Dot-matrix: pins strike an ink ribbon
- Inkjet: sprays tiny drops of ink
- Laser: toner fixed by a drum and heat
- All print dots, like a raster image

Plotters

- A pen moves over paper to draw lines
- Flatbed and drum types
- Draw like a vector display
- Used for engineering drawings and maps

3D viewing

- Stereo glasses: a different image for each eye
- VR headsets
- Covered in Week 13

Active and passive graphics

Active (interactive)

The user controls what is shown, and the picture responds.

- **Video games**
- **CAD software**
- **Flight simulators**

Passive (non-interactive)

The user only watches; they cannot change the picture.

- **TV broadcasts**
- **Printed charts**
- **Pre-rendered films**

Computer graphics software

General programming packages

Libraries you call from code to draw anything.

- **OpenGL (our labs)**
- **GKS**
- **PHIGS**
- **WebGL, DirectX, Vulkan**

Special-purpose packages

Ready-made programs used through menus, for one kind of job.

- **AutoCAD: design**
- **Photoshop: images**
- **Blender: 3D models and animation**

Part 3

Scan conversion

Turning shapes into pixels: points, lines and aliasing

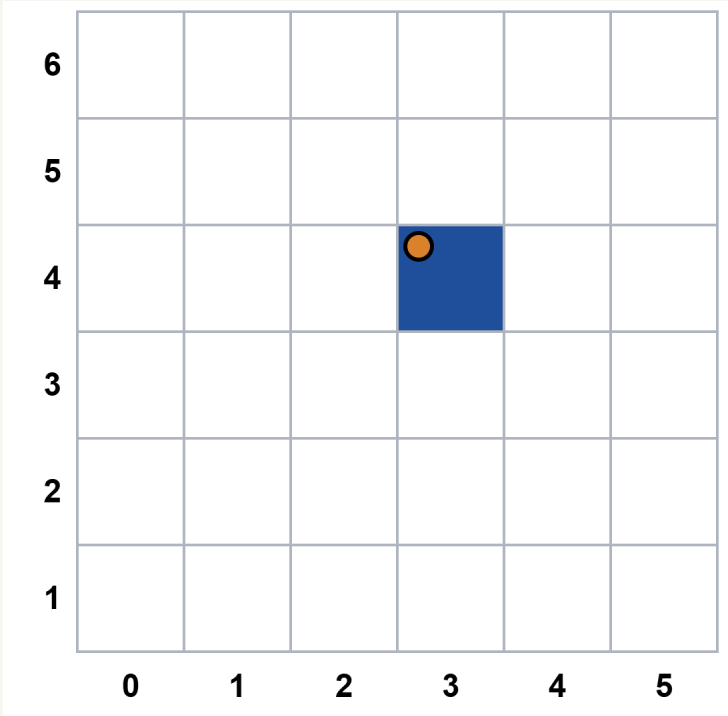
What is scan conversion?

Converting a shape from its maths description into the set of pixels that best shows it. Also called rasterization.



It is done by the graphics system or the program, for every point, line, circle and polygon.

Scan converting a point



A maths point (x, y) must become a whole-number pixel (x′, y′).

Method 1: floor

$x' = \text{floor}(x),\ y' = \text{floor}(y)$

$(2.7, 4.3) \rightarrow (2, 4)$

Method 2: round (better)

$x' = \text{floor}(x + 0.5),\ y' = \text{floor}(y + 0.5)$

$(2.7, 4.3) \rightarrow (3, 4)$

Orange dot: the true point. Blue square: the pixel chosen by rounding.

Scan converting a line: the problem

A line segment is given by its two end points (x1, y1) and (x2, y2).

Line equation: y = mx + b

- Slope: m = (y2 – y1) ÷ (x2 – x1)
- y-intercept: b = y1 – m·x1

The question

We know only the two end pixels. Which pixels in between should we turn on so the line looks as straight as possible?

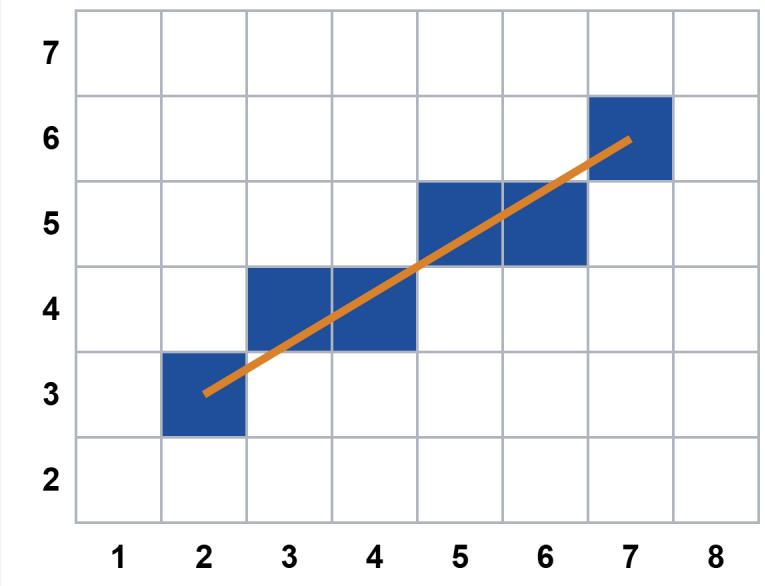
The direct method, with an example

Steps: find m and b; for each x from x1 to x2, compute y = mx + b and round it.

Line (2, 3) to (7, 6): m = 3 ÷ 5 = 0.6, b = 3 − 0.6 × 2 = 1.8

x	2	3	4	5	6	7
y = 0.6x + 1.8	3	3.6	4.2	4.8	5.4	6
Pixel y	3	4	4	5	5	6

Pixels: (2,3) (3,4) (4,4) (5,5) (6,5) (7,6)



Two problems with the direct method

1. It is slow

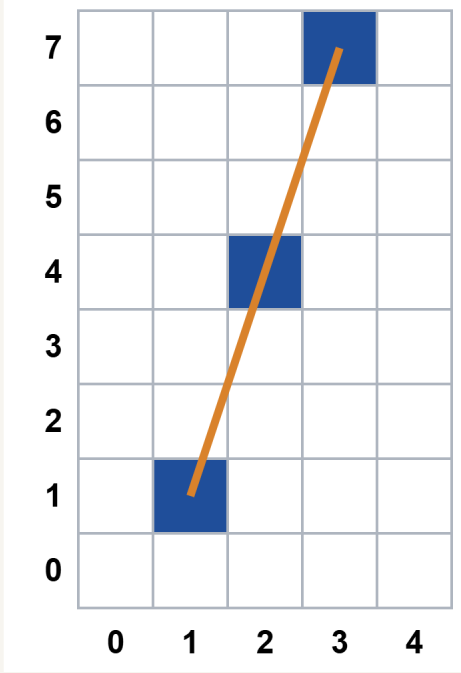
Every pixel needs a multiplication (m × x), an addition and a rounding.

2. Steep lines get gaps

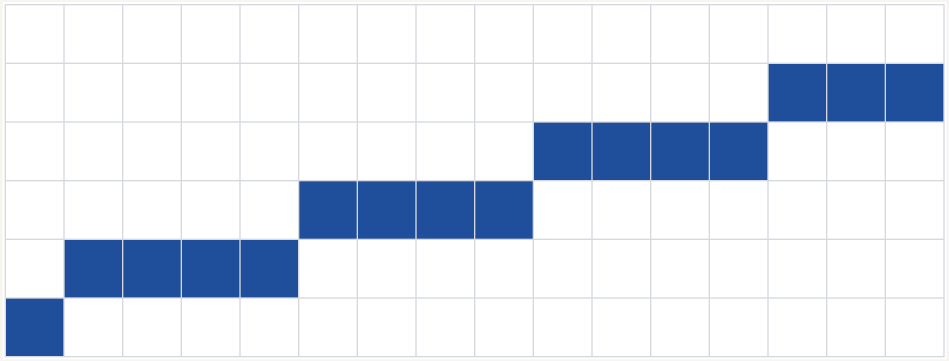
Line (1, 1) to (3, 7): m = 3. Stepping x by 1 gives only (1,1), (2,4), (3,7).

Fix: when |m| > 1, step along y instead.

Next class: the DDA algorithm solves both problems.

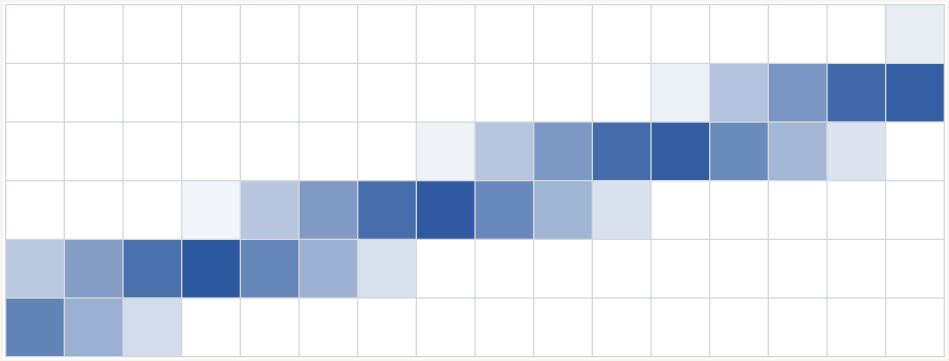


Aliasing: why lines look jagged



Aliased: the “jaggies”

Each pixel is fully on or off, so a sloped line becomes a staircase.



Anti-aliased

Edge pixels get a lighter shade depending on how much the line covers them.

Aliasing happens because a smooth shape is sampled on a grid of square pixels.

Key points from this lecture

1. RGB mixes red, green and blue light, each from 0 to 255; n bits per pixel give 2^n colours.
2. Direct coding stores RGB per pixel; a lookup table stores an index into a colour table.
3. Raster scan draws row by row; random scan draws lines; storage tubes need no refresh.
4. Scan conversion turns shapes into pixels; round with floor(x + 0.5).
5. The direct method is slow and leaves gaps on steep lines.
6. Aliasing gives jagged "staircase" edges because pixels are square.

Practice questions

1. Write the RGB values of yellow and cyan.
2. How many bytes does an 800 × 600 frame buffer need at 24 bits per pixel?
3. Give two differences between raster scan and random scan displays.
4. Scan convert the point (5.6, 2.2) using rounding.
5. Use the direct method to find the pixels of the line from (0, 0) to (4, 3).
6. Why does the direct method leave gaps on the line from (1, 1) to (3, 7)? How can we fix it?
7. What is aliasing? Give one example.

Coming up

Next lecture

The DDA line algorithm

Drawing a line by adding a small step each time: no multiplication, works for every slope

Labs

Lab 1 and Lab 2

Lab 1: set up C++ with GLUT and draw points and lines. Lab 2: code the direct method and DDA and compare them.

Questions?

Thank you

Ekramul Haque Amin · Lecturer, AI and Data Science